Star Pattern Programs

Pattern 1: Print nxn Star Square

Print a square pattern of stars (`*`) of size `n x n` .

Output

```
* * * *
* * * *
* * * *
* * * *
```

Approach:

- **Outer Loop (Rows):** Run from `i = 0` to `i = n - 1`
- **Inner Loop (Columns):** For each row, loop from `j = 0` to `j = n - 1`
- **Build Row String:** Append `*` in each inner loop iteration.
- **Print Row:**After the inner loop, print the complete row string.

Time & Space Complexity:

Time Complexity: `O(n^2)`

Space Complexity: `O(n)` (temporary row string)

JavaScript Python Java C++ C C#

```
let n = 4;
```

```
for (let i = 0; i < n; i++) {
  let row = "";
  for (let j = 0; j < n; j++) {
    row += "*";
  }
  console.log(row);
}
```

Pattern 2: Right-Angled Star Triangle

Print a right-angled triangle of stars with `n` rows.

Output

```
*
* *
* * *
* * * *
```

Approach:

- **Outer Loop (Rows):** Run a loop from `i = 0` to `i = n - 1` . Each iteration represents one row.
- **Inner Loop (Stars per Row):** For each row `i` run another loop from `j = 0` to `j = i` and append a `*` character to a string.
- **Print Row:**Print the string after the inner loop, completes for each row.

Time & Space Complexity:

Time Complexity: `O(n^2)` because the total number of stars printed is `1+2+3+.....+n = n(n+1)/2.`

Space Complexity: `O(n)` for the temporary string variable storing each row.

JavaScript Python Java C++ C C#

```
let n = 4;
for (let i = 0; i < n; i++) {
  let row = "";
  for (let j = 0; j <= i; j++) {
    row += "*";
  }
  console.log(row);
}
```

Pattern 3: Print a Right-Angled Number Triangle

Write a program that prints a right-angled triangle of numbers of height `n` .

Output

```
1
1 2
1 2 3
1 2 3 4
```

Approach:

- **Outer loop (Rows):** Run a loop from `i = 0` to `i < n` .Each iteration represents a new row.
- **Inner loop (Numbers):**Run an inner loop from `j = 0` to `j <= i` , and append `j+1` to the row.
- **Build and Print:** Construct a string for the row and print it after the inner loop ends.

Time & Space Complexity:

Time Complexity: `O(n^2)` Each row can have up to `n` numbers.

Space Complexity: `O(n)` Temporary string to build each row.

JavaScript | Python | Java | C++ | C | C#

```
let n = 4;
for (let i = 0; i < n; i++) {
  let row = "";
  for (let j = 0; j <= i; j++) {
    row += (j + 1);
  }
  console.log(row);
}
```

Pattern 4: Print a Right-Angled Triangle of Repeated Numbers

Write a program that prints a right-angled triangle where each row contains the same number repeated.

Output

```
1
2 2
3 3 3
4 4 4 4
```

Approach:

- **Outer loop (Rows):** Loop from `i = 0` to `i < n` .
- **Inner loop (Numbers):**Loop from `j = 0` to `j <= i` , appending `i+1` as a string.
- **Build and Print:** Build the row string and print it.

Time & Space Complexity:

Time Complexity: `O(n^2)`

Space Complexity: `O(n)` for the temporary row string.

JavaScript | Python | Java | C++ | C | C#

```
let n = 4;
for (let i = 0; i < n; i++) {
  let row = "";
  for (let j = 0; j <= i; j++) {
    row += (i + 1);
  }
  console.log(row);
}
```

Pattern 5: Print a Reverse Right-Angled Triangle of Increasing Numbers

Write a program that prints a reverse right-angled triangle where each row starts from 1 and the number of elements decreases with each row.

Output

```
1 2 3 4
1 2 3
1 2
1
```

Approach:

- **Outer loop (Rows):** Loop `i` from `0` to `n - 1` .Each iteration represents a row.
- **Inner loop (Print Numbers):**For each row, loop `j` from `0` to `n-i-1` and append `j+1` to a row string.
- **Print Row:** After the inner loop, print the row string.

Time & Space Complexity:

Time Complexity: `O(n^2)`

Space Complexity: `O(n)` for the temporary row string.

JavaScript | Python | Java | C++ | C | C#

```
let n = 4;
for (let i = 0; i < n; i++) {
  let row = "";
  for (let j = 0; j < n - i; j++) {
    row += (j + 1);
  }
  console.log(row);
}
```

Pattern 6: Print a Right-Aligned Right-Angled Triangle of Stars

Write a program that prints a right-aligned triangle of stars increasing row by row, with leading spaces for alignment.

Output

```
      *
    * *
  * * *
* * * *
```

Approach:

- **Outer loop (Rows):** Loop `i = 0` from `0` to `n-1` .Each iteration is a new row.
- **Inner loop 1(Spaces):**For each row, add `n - i - 1` spaces before the stars to right-align the triangle.
- **Inner loop 2(Stars):**Add `i+1` stars after the spaces.
- **Print Row:** Combine the spaces and stars, then print the row.

Time & Space Complexity:

Time Complexity: `O(n^2)`

Space Complexity: O(n) for the row string.

JavaScript | Python | Java | C++ | C | C#

```
let n = 4;
for (let i = 0; i < n; i++) {
  let row = "";
  for (let j = 0; j < n - (i + 1); j++) {
    row += " ";
  }
  for (let k = 0; k < i + 1; k++) {
    row += "*";
  }
```

```
    console.log(row);
  }
```

Pattern 7: Print a Right-Angled Triangle of Alternating 1s and 0s

Write a program that prints a triangle of alternating 1s and 0s starting with 1 on each row.

Output

```
1
1 0
1 0 1
1 0 1 0
```

Approach:

- **Outer loop (Rows):** Loop **i** from **0** to **n-1** .
- **Initialize toggle = 1:**Start each row with **toggle = 1**. .
- **Inner loop (Columns):**For each row, loop **j** from **0** to **i** . On each iteration:

Append **toggle** to the row string.

Flip **toggle** between 1 and 0.

- **Print Row:** After inner loop, print the row string.

Time & Space Complexity:

Time Complexity: O(n^2)

Space Complexity: O(n) per row

JavaScript Python Java C++ C C#

```javascript
let n = 4;
for (let i = 0; i < n; i++) {
  let row = "";
  let toggle = 1;
  for (let j = 0; j < i + 1; j++) {
    row += toggle;
    toggle = toggle === 1 ? 0 : 1;
  }
  console.log(row);
}
```

Pattern 8: Right-Angled Triangle of Alternating 1s and 0s (Global Toggle)

Write a program to print a triangle of alternating 1s and 0s, but the toggle continues globally across rows.

Output

```
1
0 1
0 1 0
1 0 1 0
```

Approach:

- **Global Toggle Variable:** Declare **toggle = 1** before the outer loop.
- **Outer Loop:**Loop **i** from **0** to **n-1** .
- **Inner loop:**Loop **j** from **0** to **i** .On each iteration:
- **Append:** **toggle** to the row string.

- **Flip:** `toggle 1 -> 0` and `0 -> 1` .
- **Print the Row:** after the inner loop.

Key Difference:

In the previous pattern, `toggle = 1` was reset each row. Here, the toggle continues globally across the entire pattern.

Time & Space Complexity:

Time Complexity: `O(n^2)`

Space Complexity: `O(n)` per row.

JavaScript | Python | Java | C++ | C | C#

```
let n = 4;
let toggle = 1;
for (let i = 0; i < n; i++) {
  let row = "";
  for (let j = 0; j < i + 1; j++) {
    row += toggle;
    toggle = toggle === 1 ? 0 : 1;
  }
  console.log(row);
}
```